DESIGN MEMO — DRY RUN, PRE-IMPLEMENTATION

Recall Architecture

How a persistent memory layer sits behind a conversational agent: what it captures, how it decides what to bring back into a reply, and what happens when a piece of it fails.

Scope: **general conversational memory** (single-user, self-service)

Status: **architecture walkthrough** — no code shipped against this yet

Prepared: **29 Aug 2026**

---

00 Scope of this document

WHAT THIS IS / ISN'T

This is a dry run — a walkthrough of the architecture before implementation starts, so the design can be argued with cheaply. It is not the milestone implementation plan (that document has separate demo commands and success criteria per milestone), and it is not the customer-support-copilot variant discussed as a possible later extension. It covers the general, single-user conversational memory system only: one person, one assistant, memory that persists across sessions.

01 End-to-end architecture

Everything above the storage line runs as LangGraph graphs behind a FastAPI gateway. One graph — the **response graph** — sits on the request's critical path and is built to

degrade, not fail: a circuit breaker guards the retrieval hop, and when it's open the graph routes straight to the model with no memory attached rather than blocking the reply. Two other graphs — **capture** and the **background jobs** — never touch the critical path at all; they run after the response has already streamed back.

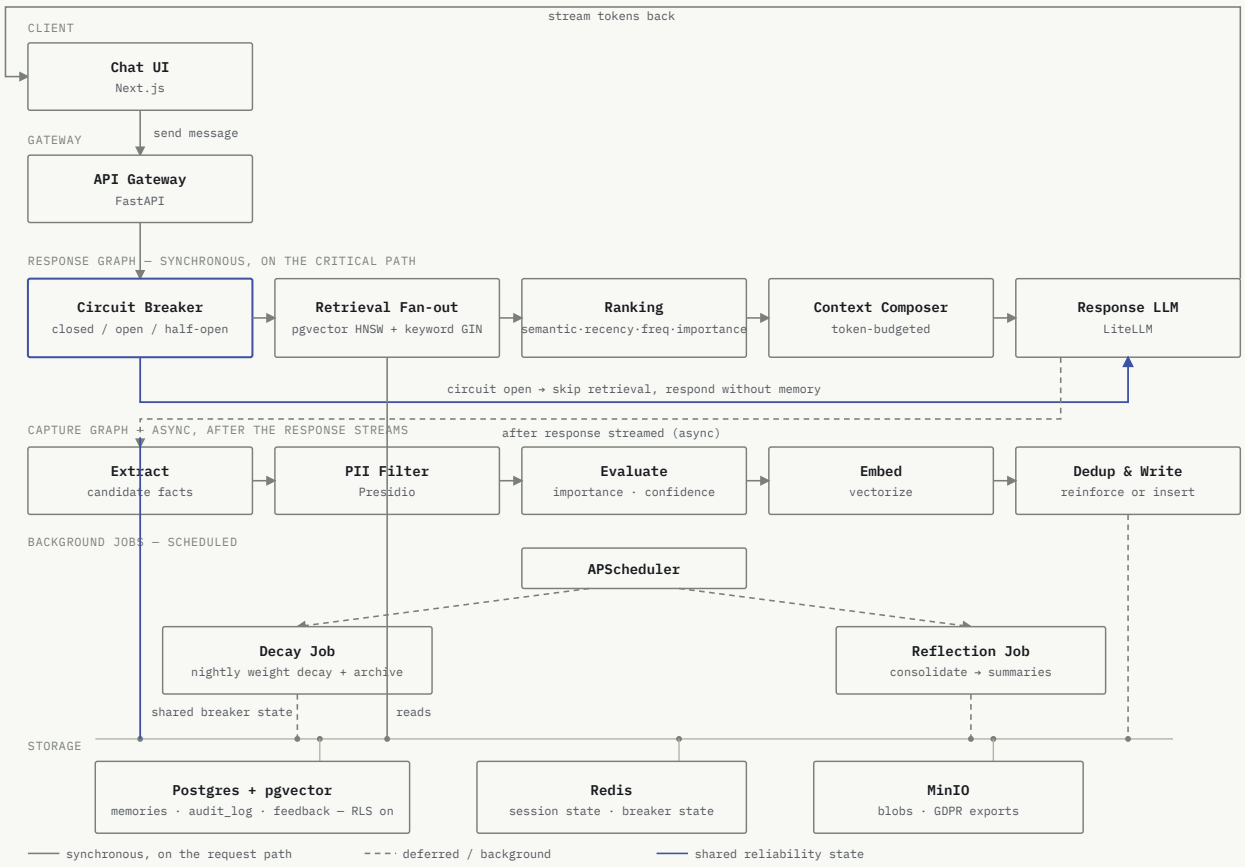

Figure 1. The response graph is the only thing on the request's critical path. Capture, decay, and reflection all write memory back into the same store, but never make a reply wait on them.

02 What happens in one turn

1. The user sends a message. The gateway opens a response-graph run and checks the circuit breaker before doing anything else.
2. **Breaker closed:** retrieval fans out to pgvector (semantic) and Postgres full-text search (keyword) in parallel, both under a per-call timeout.
3. Candidates are scored by the ranking node and the composer fills a fixed token budget, dropping the lowest-ranked memory first if there's overflow.

4. The Response LLM streams its reply back to the client — memory-augmented, but the user never waits on retrieval finishing before tokens start arriving.
5. Only after the reply has streamed does the capture graph run, asynchronously: extract candidate facts, strip PII, score importance/confidence, embed, then either reinforce an existing memory or write a new one.

If retrieval times out or the breaker is open, steps 2–3 are skipped entirely and the model answers from the conversation alone — slower or less personalized, never broken.

03 Memory model

Memory is tiered by how long it should live: **working** (this turn), **session** (this conversation), **long-term** (durable facts about the user), and **knowledge** (consolidated summaries produced by reflection). Every row that survives to long-term carries the fields below, and every retrieval ranks candidates the same way regardless of which tier they came from.

`importance` `confidence` `source` `weight` `reinforcement_count` `embedding` `content_tsv` `deleted_at`

```
score = 0.4·semantic + 0.2·recency + 0.2·frequency + 0.2·importance
```

FORWARD-COMPATIBLE SEAM

The schema separates `subject_id` (whose memory this is) from `actor_id` (who's reading or writing it). For this general assistant the two are always equal — you and the owner of your own memories are the same person — so it costs nothing today. It's there because collapsing them into a single `user_id` now and splitting them apart later would mean an RLS rewrite and a data migration, and a future extension (an agent acting on behalf of many people, e.g. a support copilot) needs exactly that split to exist at all.

04 Reliability: the circuit breaker

Retrieval is the one hop in the request path that talks to another service, so it's the one thing guarded by a breaker. State lives in Redis rather than in a process, because a fallback that only one replica knows about isn't a fallback — every replica has to see the same open/closed state at once.

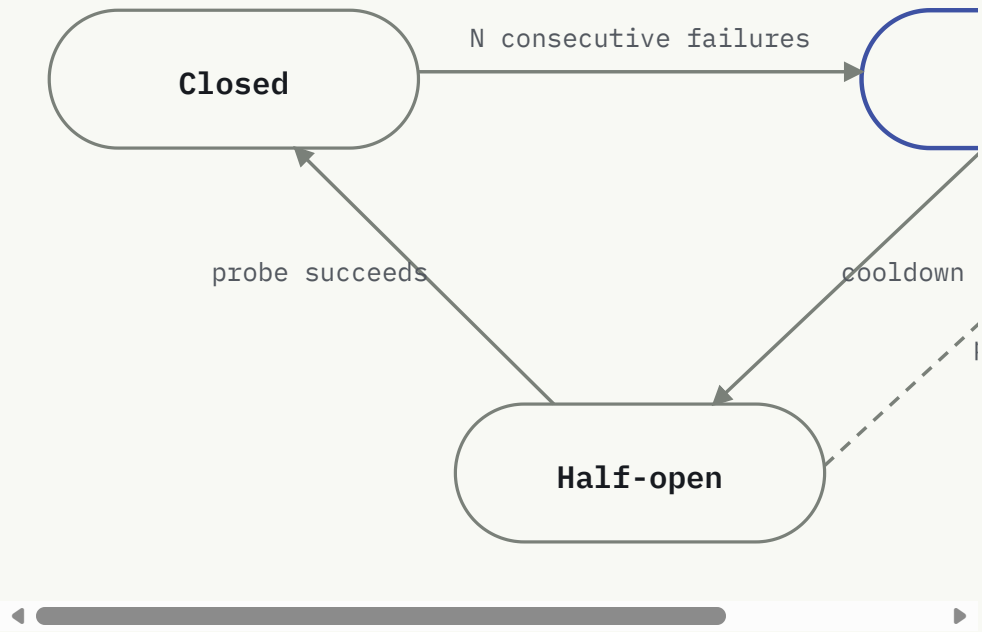

Figure 2. Only the half-open probe actually calls retrieval while degraded — everything else in the open state is a cheap Redis read.

05 Governance

Every write, read, and delete against a memory appends one row to an append-only audit log — not for the user, for whoever has to answer "who accessed this" later. Deletes are soft: a `deleted_at` timestamp, filtered out of both retrieval and the user-facing memory list, but retained (marked as deleted) in a separate GDPR export endpoint that returns the full record of what's ever been stored, distinct from the curated view a user browses day to day.

06 Evaluation loop

A small labeled set of query → expected-memory pairs is the thing that actually gets checked before retrieval changes ship: precision and recall against that golden set, with fixtures that force a keyword-only match and a semantic-only match so a passing run proves both retrieval paths ran and were merged — not that one of them silently carried the whole score.

07 Technology choices

LAYER	TECHNOLOGY	WHY THIS, NOT THE OBVIOUS ALTERNATIVE
Orchestration	LangGraph	Explicit graph structure for the breaker/fallback branch — a linear chain can't express "skip this hop and continue" as cleanly.
Model access	LiteLLM	One call surface across providers, so the response model and the embedding model are each a config value, not a rewrite.
Vector + keyword	Postgres + pgvector	Hybrid search in one transactionally-consistent store instead of a vector DB kept in sync with a separate keyword index.
Session / breaker state	Redis	Sub-millisecond shared state that every replica reads the same way — the one thing a circuit breaker actually needs.
Isolation	Postgres RLS	Enforced at the database, not in application code — a missed WHERE user_id = can't leak another user's memory.
PII removal	Presidio	Runs before a memory is ever written, not as a redaction pass after the fact.
Scheduling	APScheduler	LangGraph has no native cron; decay and reflection are triggered externally, not polled for.
Object storage	MinIO	S3-compatible target for blobs and GDPR export bundles, swappable for real S3 without a code change.

08 Why this shape, not a simpler one

Three shapes were on the table before any of the above got written down.

A

CHOSEN

Full stack

Postgres+pgvector, Redis, LangGraph, LiteLLM, hybrid retrieval, breaker, governance — the real target architecture, built as itself.

Costs more before the first demo, but every later milestone ships the actual system rather than a throwaway version of it.

B

Stripped prototype

Keyword-only search, in-process session state, no breaker. Fastest to a first working loop.

Validates the write/read cycle, not the reliability or hybrid-retrieval behavior the design actually depends on — largely a rebuild to reach A.

C

Managed memory SaaS

Delegate storage and retrieval to a third-party memory API. Least infrastructure to run.

Vendor-locks the one layer — storage/retrieval — that most needs to stay swappable, and rules out enforcing isolation with RLS.

Next: front-load a thin end-to-end slice before the reliability and governance milestones